Scrapy Data Extraction and Analysis Project

YZV 322E – Applied Data Engineering
Individual Tool Presentation

Gonca Kaplan
Student ID: 150220324

Spring 2026

Tool: Scrapy

A Python web scraping framework for ETL data ingestion

What is Scrapy?

- Scrapy is an open-source Python framework for web crawling and web scraping.
- It extracts structured data from websites.
- Output formats include JSON, CSV, XML, and other feed exports.
- It is commonly used for data mining, monitoring, automation, and ETL ingestion.

Basic information

- First released: 2008
- Maintainer: Scrapy community / Zyte ecosystem
- License: BSD License

Main Idea

Scrapy turns unstructured web pages into structured datasets.

What Problem Does It Solve?

Without Scrapy

- Manual copy-paste is slow.
- Data can be inconsistent.
- Large websites are hard to collect manually.
- Pagination makes extraction repetitive.

With Scrapy

- Automated crawling
- CSS/XPath based extraction
- Pagination support
- Structured output for analysis

Motivation

In data engineering, reliable data collection is the first step before storage, transformation, analysis, or orchestration.

Scrapy vs Alternatives

Tool	Strength	Limitation
Scrapy	Full crawling framework	More setup than simple scripts
BeautifulSoup	Easy HTML parsing	No built-in crawling system
Selenium	Handles JavaScript pages	Slower and heavier

- Scrapy is best when we need structured extraction from many pages.
- BeautifulSoup is better for small one-page parsing tasks.
- Selenium is useful when the website depends heavily on browser interaction.

Course Connection: Python ETL

Scrapy → JSON Dataset → Pandas Analysis → Visual Output

- Scrapy is listed as a Python ETL-related tool.
- It acts as the **Extract** step of an ETL pipeline.
- The scraped data can later be loaded into PostgreSQL.
- It can also be scheduled with Airflow for repeated collection.

Pipeline Role

Scrapy is not only a scraping tool; it is a data ingestion component in a larger data engineering workflow.

Setup and Repository Structure

Installation

```
pip install -r requirements.txt
```

Run the scraper

```
python -m scrapy crawl books -O output.json
```

Run the analysis

```
python analysis.py
```

Repository structure

```
scrapy_project/
|-- my_scraper/
|   |-- spiders/
|   |   |-- books.py
|-- analysis.py
|-- output.json
|-- plots/
|-- requirements.txt
|-- README.md
```

Demo: Scrapy Spider

```
class BooksSpider(scrapy.Spider):
    name = "books"
    start_urls = ["http://books.toscrape.com/"]

    def parse(self, response):
        for book in response.css("article.product_pod"):
            rating = book.css("p.star-rating::attr(class)").get()
            rating = rating.split()[-1]

            yield {
                "title": book.css("h3 a::attr(title)").get(),
                "price": book.css(".price_color::text").get(),
                "rating": rating,
            }

        next_page = response.css("li.next a::attr(href)").get()
        if next_page:
            yield response.follow(next_page, callback=self.parse)
```

- Extracts title, price, and rating
- Follows pagination automatically

Demo Output

Scraping result

- Total scraped books: **1000**
- Pages crawled: **50**
- Output file: `output.json`
- Spider finished successfully

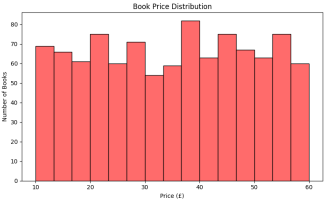
Example JSON item

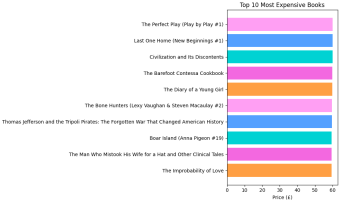
```
{
  "title":  "A Light in the Attic",
  "price":  "£51.77",
  "rating":  "Three"
}
```

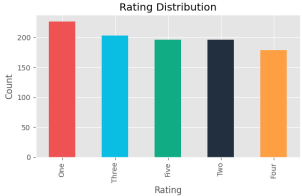
Observation

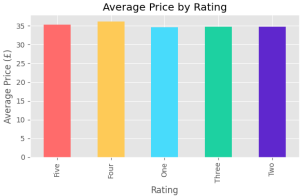
The result is machine-readable and ready for analysis.

Data Analysis and Visualizations









Scraped data was analyzed with Pandas and visualized with Matplotlib.

Conclusion

Key Takeaways

- Scrapy enables automated and scalable data extraction from websites.
- It significantly reduces manual effort and improves data consistency.
- Extracted data can be easily transformed into structured formats like JSON.
- The project demonstrates a simple yet effective ETL pipeline.

What I Learned

- How to use Scrapy for real-world data extraction
- Handling pagination and structured data parsing
- Converting raw data into meaningful insights using Python

Thank You!